PYTHON ACTIVITIES BASED LEARNING

10 Case Studies — Complete Documentation

Name: N. Lakshmirasnaath

Roll Number: 2600 09 63

Branch: CSM

Section: 2

Each case study contains a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output screenshot instructions, and conclusion.

Important: Flowcharts are provided as readable step-by-step flow layouts. The output screenshot section tells you to run the code and capture the real output in VS Code. Check all case-study names and rules against the exact slides from your lecturer before submitting.

Case Study 1: Progressive Electricity Billing

1. Problem Statement

Calculate an electricity bill using progressive slabs: first 100 units at Rs. 1.50 per unit, next 100 units at Rs. 2.50 per unit, and units above 200 at Rs. 4.00 per unit.

2. Objectives

• Read electricity units.

• Apply the correct progressive slabs.

• Display the total bill.

3. Assumptions

• Units must be zero or greater.

• Rates are example rates for this assignment, not an official tariff.

4. Algorithm

1. Start.

2. Input units.

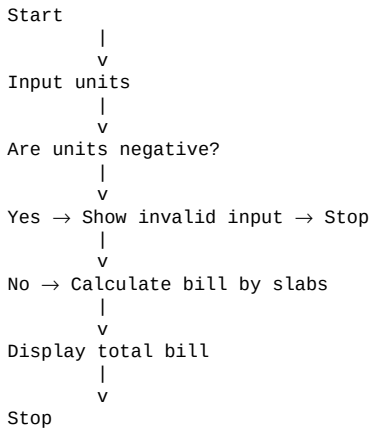
3. If units are negative, show an error.

4. Otherwise calculate the charge using the applicable slabs.

5. Display the bill.

6. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT units
IF units < 0
    PRINT invalid input
ELSE IF units <= 100
    bill = units * 1.50
ELSE IF units <= 200
    bill = 150 + (units - 100) * 2.50
ELSE
    bill = 400 + (units - 200) * 4.00
PRINT bill if units >= 0
STOP
```

7. Python Source Code

```
units = int(input("Enter electricity units: "))

if units < 0:
    print("Invalid units")
elif units <= 100:
    bill = units * 1.50
elif units <= 200:
    bill = 150 + (units - 100) * 2.50
else:
    bill = 400 + (units - 200) * 4.00

if units >= 0:
    print(f"Total bill: Rs. {bill:.2f}")
```

8. Test Cases and Expected Results

Input	Expected output
80	Rs. 120.00
150	Rs. 275.00
250	Rs. 600.00
-5	Invalid units

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program calculates the electricity bill by applying the correct progressive rate slabs.

Case Study 2: Tuple-Based Transaction Audit

1. Problem Statement

Store transactions as tuples, calculate their total value, and display transactions that exceed a user-entered audit limit.

2. Objectives

• Represent transactions using tuples.

• Calculate the total transaction value.

• Identify transactions above the limit.

3. Assumptions

• Transaction amounts are non-negative.

• Sample transactions are used for demonstration.

4. Algorithm

1. Start.

2. Create a tuple of transaction ID and amount pairs.

3. Input the audit limit.

4. Add each transaction amount to the total.

5. Display any transaction above the limit.

6. Display the total and stop.

5. Flowchart

```
Start
        |
        v
Create transaction tuples
        |
        v
Input audit limit
        |
        v
Loop through transactions
        |
        v
Add amount to total
        |
        v
Is amount > limit? → Yes: display transaction
        |
        v
More transactions? → Repeat
        |
        v
Display total
        |
        v
Stop
```

6. Pseudocode

```
START
transactions = ((ID, amount), ...)
INPUT limit
total = 0
FOR each transaction
    total = total + amount
    IF amount > limit
        DISPLAY transaction
DISPLAY total
STOP
```

7. Python Source Code

```
transactions = (
    ("T101", 450.00),
    ("T102", 1800.00),
    ("T103", 950.00),
    ("T104", 2500.00)
)

limit = float(input("Enter audit limit: Rs. "))
total = 0

for transaction_id, amount in transactions:
```

```
    total += amount
    if amount > limit:
        print(transaction_id, "Rs.", amount)

print(f"Total transaction value: Rs. {total:.2f}")
```

8. Test Cases and Expected Results

Input limit	Expected output
1000	T102 and T104; total Rs. 5700.00
2000	T104; total Rs. 5700.00
3000	No flagged transaction; total Rs. 5700.00

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program uses tuples to audit transaction records and identify amounts that exceed a specified limit.

Case Study 3: Hospital Ward Occupancy Monitoring

1. Problem Statement

Monitor hospital ward occupancy by calculating available beds and occupancy percentages.

2. Objectives

• Store ward bed data.

• Calculate available beds and occupancy rate.

• Identify wards that are full.

3. Assumptions

• Total beds must be greater than zero.

• Occupied beds must be between zero and total beds.

• Data is fictional for practice.

4. Algorithm

1. Start.

2. Create ward records.

3. For each ward validate the bed counts.

4. Calculate available beds and occupancy percentage.

5. Display ward status and identify full wards.

6. Stop.

5. Flowchart

```
Start
        |
        v
Read ward records
        |
        v
Validate total and occupied beds
        |
        v
Calculate available beds and occupancy %
        |
        v
Display ward details
        |
        v
Are available beds zero? → Yes: display FULL
        |
        v
More wards? → Repeat
        |
        v
Stop
```

6. Pseudocode

```
START
wards = [(name, total, occupied), ...]
FOR each ward
    available = total - occupied
    percentage = occupied / total * 100
    DISPLAY details
    IF available == 0
        DISPLAY FULL
STOP
```

7. Python Source Code

```
wards = [
    ("Medical", 40, 32),
    ("Surgical", 30, 30),
    ("Children", 20, 12)
]

for name, total, occupied in wards:
    if total <= 0 or occupied < 0 or occupied > total:
        print("Invalid data for", name)
        continue

    available = total - occupied
```

```
rate = occupied / total * 100
print(f"{name}: {occupied}/{total} occupied")
print("Available beds:", available)
print(f"Occupancy: {rate:.1f}%")
if available == 0:
    print("Status: FULL")
```

8. Test Cases and Expected Results

Ward data	Expected output
Medical: 32/40	8 beds available; 80.0%
Surgical: 30/30	0 available; 100.0%; FULL
Children: 12/20	8 available; 60.0%

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program summarizes ward occupancy and shows how many beds remain available.

Case Study 4: University Examination Result Moderation

1. Problem Statement

Validate student marks, calculate total and average, and assign a result using a simple sample pass rule.

2. Objectives

• Accept marks for multiple subjects.

• Validate marks from 0 to 100.

• Calculate total, average, and pass/fail status.

3. Assumptions

• Each mark is between 0 and 100.

• A student passes only if every subject mark is at least 40.

• The rule is for practice; official university rules may differ.

4. Algorithm

1. Start.

2. Input marks separated by spaces.

3. Check that at least one mark was entered and all marks are in range.

4. Calculate total and average.

5. If all marks are at least 40, display PASS; otherwise display FAIL.

6. Stop.

5. Flowchart

```
Start
        |
        v
Input subject marks
        |
        v
Are marks present and all between 0 and 100?
        |
        v
No → Display invalid input → Stop
        |
        v
Yes → Calculate total and average
        |
        v
Are all marks ≥ 40? → Yes: PASS / No: FAIL
        |
        v
Display result
        |
        v
Stop
```

6. Pseudocode

```
START
INPUT marks
IF marks empty OR any mark < 0 OR > 100
    DISPLAY invalid marks
ELSE
    total = SUM(marks)
    average = total / count(marks)
    IF every mark >= 40: result = PASS
    ELSE: result = FAIL
    DISPLAY total, average, result
STOP
```

7. Python Source Code

```
marks_text = input("Enter marks separated by spaces: ")
marks = [int(x) for x in marks_text.split()]

if not marks:
    print("Enter at least one mark")
elif any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks: enter values from 0 to 100")
else:
    total = sum(marks)
    average = total / len(marks)
```

```
result = "PASS" if all(mark >= 40 for mark in marks) else "FAIL"
print("Total:", total)
print(f"Average: {average:.2f}")
print("Result:", result)
```

8. Test Cases and Expected Results

Input marks	Expected output
80 70 90	Total 240; average 80.00; PASS
80 35 90	Total 205; average 68.33; FAIL
70 105 80	Invalid marks
(empty)	Enter at least one mark

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program validates marks and produces a clear result summary using a stated sample rule.

Case Study 5: Student Academic Risk Calculation

1. Problem Statement

Classify a student's academic risk based on attendance and average marks using sample thresholds.

2. Objectives

• Read attendance and average marks.

• Validate values from 0 to 100.

• Classify academic risk as high, medium, or low.

3. Assumptions

• Attendance and average marks range from 0 to 100.

• High risk: attendance below 65 or average below 40.

• Medium risk: attendance below 75 or average below 50; otherwise low. These thresholds are illustrative.

4. Algorithm

1. Start.

2. Input attendance and average marks.

3. Validate both values.

4. If attendance < 65 or average < 40, classify HIGH.

5. Else if attendance < 75 or average < 50, classify MEDIUM.

6. Otherwise classify LOW.

7. Display category and stop.

5. Flowchart

```
Start
        |
        v
Input attendance and average
        |
        v
Are both values in 0–100?
        |
        v
No → Display invalid input → Stop
        |
        v
Yes → Attendance < 65 OR average < 40? → Yes: HIGH
        |
        v
Otherwise attendance < 75 OR average < 50? → Yes: MEDIUM
        |
        v
Otherwise: LOW
        |
        v
Display risk
        |
        v
Stop
```

6. Pseudocode

```
START
INPUT attendance, average
IF either value outside 0..100
    DISPLAY invalid input
ELSE IF attendance < 65 OR average < 40
    risk = HIGH
ELSE IF attendance < 75 OR average < 50
    risk = MEDIUM
ELSE
    risk = LOW
DISPLAY risk
STOP
```

7. Python Source Code

```
attendance = float(input("Enter attendance percentage: "))
average = float(input("Enter average marks: "))

if not (0 <= attendance <= 100 and 0 <= average <= 100):
```

```
    print("Invalid input: values must be from 0 to 100")
elif attendance < 65 or average < 40:
    print("Academic risk: HIGH")
elif attendance < 75 or average < 50:
    print("Academic risk: MEDIUM")
else:
    print("Academic risk: LOW")
```

8. Test Cases and Expected Results

Attendance, average	Expected output
80, 75	LOW
70, 55	MEDIUM
60, 80	HIGH
90, 35	HIGH
110, 60	Invalid input

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program demonstrates a basic academic risk category using attendance and marks.

Case Study 6: Food Supply Chain Stock Reconciliation

1. Problem Statement

Compare recorded stock with physically counted stock and identify shortages, surpluses, or matches.

2. Objectives

• Store recorded and physical quantities.

• Calculate the difference.

• Display stock status for each item.

3. Assumptions

• Stock quantities are non-negative whole numbers.

• Difference is physical stock minus recorded stock.

4. Algorithm

1. Start.

2. Create stock records.

3. For each item calculate physical minus recorded quantity.

4. If difference is zero, mark MATCH.

5. If negative, mark SHORTAGE; otherwise mark SURPLUS.

6. Display each result and stop.

5. Flowchart

```
Start
        |
        v
Read stock records
        |
        v
Calculate physical – recorded
        |
        v
Is difference = 0? → MATCH
        |
        v
If not, is difference < 0? → SHORTAGE
        |
        v
Otherwise → SURPLUS
        |
        v
Display item status
        |
        v
More items? → Repeat / Stop
```

6. Pseudocode

```
START
stock = [(item, recorded, physical), ...]
FOR each item
    difference = physical - recorded
    IF difference == 0: status = MATCH
    ELSE IF difference < 0: status = SHORTAGE
    ELSE: status = SURPLUS
    DISPLAY item, difference, status
STOP
```

7. Python Source Code

```
stock = [
    ("Rice", 100, 96),
    ("Wheat", 80, 80),
    ("Lentils", 50, 57)
]

for item, recorded, physical in stock:
    difference = physical - recorded
    if difference == 0:
        status = "MATCH"
    elif difference < 0:
        status = "SHORTAGE"
    else:
        status = "SURPLUS"
```

```
print(f"{item}: difference {difference}, status {status}")
```

8. Test Cases and Expected Results

Recorded, physical	Expected output
Rice: 100, 96	Difference -4; SHORTAGE
Wheat: 80, 80	Difference 0; MATCH
Lentils: 50, 57	Difference +7; SURPLUS

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program compares stock records and identifies differences that may need checking.

Case Study 7: Hackathon Team Formation and Ranking

1. Problem Statement

Group participants into teams of two, calculate each team's combined score, and rank teams from highest to lowest.

2. Objectives

• Group participants into pairs.

• Calculate team totals.

• Display team ranking.

3. Assumptions

• Scores are sample numeric values.

• Participants are grouped in the order listed, two per team.

• The number of participants is even in this example.

4. Algorithm

1. Start.

2. Create participant names and scores.

3. Group consecutive participants into pairs.

4. Calculate each pair's total score.

5. Sort teams by total score in descending order.

6. Display ranking and stop.

5. Flowchart

```
Start
        |
        v
Read participant names and scores
        |
        v
Group participants into pairs
        |
        v
Calculate team scores
        |
        v
Sort scores from highest to lowest
        |
        v
Display rank and team
        |
        v
Stop
```

6. Pseudocode

```
START
participants = [(name, score), ...]
CREATE pairs of participants
FOR each pair
    team_total = sum(member scores)
SORT teams by team_total descending
DISPLAY rank, team, score
STOP
```

7. Python Source Code

```
participants = [
    ("Asha", 88), ("Ravi", 82),
    ("Meena", 95), ("Kiran", 91),
    ("John", 78), ("Sara", 84)
]

teams = []
for i in range(0, len(participants), 2):
    members = participants[i:i + 2]
    total = sum(score for name, score in members)
    teams.append((total, members))

teams.sort(reverse=True, key=lambda team: team[0])

for rank, (total, members) in enumerate(teams, start=1):
    names = ", ".join(name for name, score in members)
```

```
    print(f"Rank {rank}: {names} - total score {total}")
```

8. Test Cases and Expected Results

Team	Expected output
Asha and Ravi	Total 170
Meena and Kiran	Total 186; rank 1
John and Sara	Total 162; rank 3

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program groups participants, calculates team scores, and ranks teams using sorting.

Case Study 8: Campus Transport Allocation

1. Problem Statement

Allocate students to buses without exceeding bus capacity and report any students who cannot be allocated.

2. Objectives

• Read the number of students.

• Allocate students according to bus capacities.

• Report assigned students and unused seats.

3. Assumptions

• Student count and capacities are non-negative integers.

• Students are allocated in bus-list order.

• Routes and special transport needs are outside this sample.

4. Algorithm

1. Start.

2. Input student count.

3. If negative, show an error.

4. Use the listed bus capacities.

5. For each bus assign the smaller of remaining students and bus capacity.

6. Display assigned students and empty seats.

7. Report any unallocated students and stop.

5. Flowchart

```
Start
        |
        v
Input number of students
        |
        v
Is count negative? → Yes: error → Stop
        |
        v
No → Set remaining students
        |
        v
For each bus: assign min(remaining, capacity)
        |
        v
Update remaining and display seats
        |
        v
Are students remaining? → Yes: display unallocated
        |
        v
Stop
```

6. Pseudocode

```
START
INPUT students
capacities = [40, 35, 30]
IF students < 0: DISPLAY invalid
ELSE
    remaining = students
    FOR each capacity
        assigned = MIN(remaining, capacity)
        remaining = remaining - assigned
        DISPLAY assigned, empty seats
    DISPLAY remaining unallocated
STOP
```

7. Python Source Code

```
students = int(input("Enter number of students: "))
capacities = [40, 35, 30]

if students < 0:
    print("Invalid student count")
else:
    remaining = students
```

```
for number, capacity in enumerate(capacities, start=1):
    assigned = min(remaining, capacity)
    remaining -= assigned
    print(f"Bus {number}: assigned {assigned}, empty seats {capacity - assigned}")

if remaining > 0:
    print("Students not allocated:", remaining)
else:
    print("All students allocated")
```

8. Test Cases and Expected Results

Student count	Expected output
50	Bus assignments 40, 10, 0; all allocated
105	Assignments 40, 35, 30; all allocated
120	105 assigned; 15 unallocated
-2	Invalid student count

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program assigns students within bus capacities and reports any remaining students.

Case Study 9: Multi-Branch Retail Inventory Transfer

1. Problem Statement

Compare stock and target stock at retail branches, then suggest transfers from surplus branches to branches with shortages.

2. Objectives

• Calculate branch surplus and shortage.

• Suggest transfers between branches.

• Report any shortage left after transfers.

3. Assumptions

• One product is considered across branches.

• Stock and target quantities are non-negative integers.

• Suggested transfers do not include shipping costs or approval rules.

4. Algorithm

1. Start.

2. Create branch stock and target records.

3. Separate branches with surplus and shortage.

4. For each shortage, transfer available stock from surplus branches.

5. Update surplus and shortage quantities.

6. Display suggested transfers and remaining shortage.

7. Stop.

5. Flowchart

```
Start
        |
        v
Read branch stock and target
        |
        v
Calculate stock – target
        |
        v
Separate surplus and shortage branches
        |
        v
Choose donor and receiver
        |
        v
Transfer minimum available amount
        |
        v
Update quantities
        |
        v
More shortage and surplus? → Repeat
        |
        v
Display transfers and remaining shortage
        |
        v
Stop
```

6. Pseudocode

```
START
branches = [(name, stock, target), ...]
BUILD surplus list and shortage list
FOR each receiver with shortage
    FOR each donor with surplus
        amount = MIN(receiver shortage, donor surplus)
        RECORD transfer
        UPDATE quantities
DISPLAY transfers and any remaining shortage
STOP
```

7. Python Source Code

```
branches = [
    {"name": "Branch A", "stock": 50, "target": 30},
    {"name": "Branch B", "stock": 10, "target": 25},
```

```
    {"name": "Branch C", "stock": 40, "target": 35}
]

surplus = []
shortage = []

for branch in branches:
    difference = branch["stock"] - branch["target"]
    if difference > 0:
        surplus.append([branch["name"], difference])
    elif difference < 0:
        shortage.append([branch["name"], -difference])

for receiver in shortage:
    for donor in surplus:
        amount = min(receiver[1], donor[1])
        if amount > 0:
            print(f"Transfer {amount} units: {donor[0]} -> {receiver[0]}")
            receiver[1] -= amount
            donor[1] -= amount
        if receiver[1] == 0:
            break

for receiver in shortage:
    if receiver[1] > 0:
        print(f"Remaining shortage at {receiver[0]}: {receiver[1]} units")
```

8. Test Cases and Expected Results

Branch stock/target	Expected output
A 50/30; B 10/25; C 40/35	Transfer 15 units from A to B; 5 surplus units remain at A and 5 at C
All at target	No transfers suggested
Total surplus < shortage	Available surplus transferred; remaining shortage displayed

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program demonstrates a simple inventory balancing method across branches.

Case Study 10: Integrated E-Commerce Order Processing and Reconciliation

1. Problem Statement

Process sample online orders by validating products and quantities, checking stock, calculating order totals, and updating inventory for accepted orders.

2. Objectives

• Validate order details.

• Check stock availability.

• Calculate totals and update inventory.

• Summarize accepted and rejected orders.

3. Assumptions

• Orders request one product and a positive quantity.

• Product prices and inventory are sample data.

• Tax, shipping, discounts, and payment gateway operations are excluded.

4. Algorithm

1. Start.

2. Create product price and stock records.

3. Read the order list.

4. For each order validate product and quantity.

5. If stock is sufficient, calculate total, reduce stock, and accept order.

6. Otherwise reject the order with a reason.

7. Display order statuses, counts, and final stock.

8. Stop.

5. Flowchart

```
Start
        |
        v
Create product inventory and orders
        |
        v
Read next order
        |
        v
Is product valid and quantity positive?
        |
        v
No → Reject order
        |
        v
Yes → Is stock sufficient?
        |
        v
No → Reject for insufficient stock
        |
        v
Yes → Calculate total, update stock, accept order
        |
        v
More orders? → Repeat
        |
        v
Display summary and final stock
        |
        v
Stop
```

6. Pseudocode

```
START
products = product -> (price, stock)
orders = [(order_id, product, quantity), ...]
accepted = 0; rejected = 0
FOR each order
```

```
    IF product invalid OR quantity <= 0: reject
    ELSE IF stock < quantity: reject
    ELSE
        total = price * quantity
        stock = stock - quantity
        accept
    DISPLAY status
DISPLAY counts and final stock
STOP
```

7. Python Source Code

```
products = {
    "Notebook": {"price": 50.0, "stock": 10},
    "Pen": {"price": 10.0, "stock": 25},
    "Bag": {"price": 600.0, "stock": 3}
}

orders = [
    ("ORD101", "Notebook", 2),
    ("ORD102", "Bag", 4),
    ("ORD103", "Pen", 5),
    ("ORD104", "Pencil", 2)
]

accepted = 0
rejected = 0

for order_id, product, quantity in orders:
    if quantity <= 0 or product not in products:
        print(order_id, "REJECTED: invalid product or quantity")
        rejected += 1
    elif products[product]["stock"] < quantity:
        print(order_id, "REJECTED: insufficient stock")
        rejected += 1
    else:
        total = products[product]["price"] * quantity
        products[product]["stock"] -= quantity
        print(f"{order_id} ACCEPTED: total Rs. {total:.2f}")
        accepted += 1

print("Accepted orders:", accepted)
print("Rejected orders:", rejected)
print("Final stock:")
for product, details in products.items():
    print(product, details["stock"])
```

8. Test Cases and Expected Results

Order	Expected output
ORD101 Notebook × 2	Accepted; Rs. 100.00; stock becomes 8
ORD102 Bag × 4	Rejected: insufficient stock
ORD103 Pen × 5	Accepted; Rs. 50.00; stock becomes 20
ORD104 Pencil × 2	Rejected: invalid product
Summary	2 accepted; 2 rejected

9. Output Screenshots

Run this program in VS Code, then take a screenshot of the terminal showing the output. Save the screenshot in this case study's folder. The expected results above are for checking your run.

10. Conclusion

The program performs basic order validation, stock checks, total calculation, and inventory reconciliation.